

LEVEL 5: Sorting Books

1. Objective

Arrange books by Book ID so that the records become ordered and can later be searched efficiently.

2. Data Structure / Algorithm Used

Array of Structures + Bubble Sort

3. Problem Statement

Extend the Library Management System by implementing the sorting books module. Complete this module independently before integrating it with previous levels.

4. Algorithm / Working

Compare adjacent Book IDs → swap when the left ID is larger → repeat passes until the array is sorted.

5. C Program

```
#include <stdio.h>
typedef struct { int id; char title[40]; } Book;

void sort(Book b[], int n) {
    int i, j; Book t;
    for(i=0; i<n-1; i++)
        for(j=0; j<n-1-i; j++)
            if(b[j].id > b[j+1].id) {
                t = b[j]; b[j] = b[j+1]; b[j+1] = t;
            }
}

int main() {
    Book b[] = {{105, "Data Structures"}, {101, "C Programming"},
                {103, "Operating System"}};
    int i, n = 3;
    sort(b, n);
    for(i=0; i<n; i++) printf("%d %s\n", b[i].id, b[i].title);
    return 0;
}
```

6. Sample Input

Initial IDs: 105, 101, 103

7. Sample Output

101 C Programming
103 Operating System
105 Data Structures

8. Complexity

Standard Bubble Sort: $O(n^2)$ time and $O(1)$ extra space.

9. Practice Exercises

- Sort descending.
- Sort by title.
- Implement optimized Bubble Sort.

10. Related Viva Questions

- What is Bubble Sort?
- Why can a complete structure be swapped?
- What is its worst-case complexity?
- Why sort before Binary Search?